

Objetivo:

Implementación de un programa sencillo en C++ para gestionar una agenda de contactos, utilizando un diseño basado en TADs.

Descripción detallada:

- a) Implementa el TAD contacto definido como:

```

espec contactos
  usa cadenas, enteros, booleanos
  género contacto {Los valores del TAD representan personas, que llamamos contactos, para
    las que se tiene información de su nombre, su dirección y su número de teléfono}
operaciones
  crear: cadena nom, cadena dir, entero tel -> contacto
    {Dada una cadena nom, una cadena dir, y un entero tel, se obtiene un contacto
    con nombre nom, dirección dir y número de teléfono tel}
  nombre: contacto c -> cadena
    {Dado un contacto c, se obtiene la cadena correspondiente al nombre de c}
  dirección: contacto c -> cadena
    {Dado un contacto c, se obtiene la cadena correspondiente a la dirección de c}
  teléfono: contacto c -> entero
    {Dado un contacto c, se obtiene el entero correspondiente al teléfono de c}
  iguales: contacto c1, contacto c2 -> booleano
    {Devuelve verdad si y sólo los contactos c1 y c2 tienen el mismo nombre}
fespec

```

- b) Implementa el TAD agenda definido a continuación añadiendo además las operaciones necesarias para dotar al TAD de un iterador¹.

```

espec agendas
  usa contactos, cadenas, booleanos
  género agenda {Los valores del TAD representan colecciones de contactos a las
    que se pueden añadir elementos de tipo contacto, y de las que se pueden eliminar
    sus contactos de uno en uno, eliminándose siempre el último contacto añadido de
    todos los que contenga la agenda}
operaciones
  iniciar: -> agenda
    {Devuelve una agenda vacía, sin contactos}
  añadir: agenda a, contacto c -> agenda
    {Devuelve una agenda igual a la resultante de añadir un contacto c a la agenda a.}
  vacía: agenda a -> booleano
    {Devuelve verdad si y sólo si la agenda a está vacía}
  borrarUltimo: agenda a -> agenda
    {Si a no está vacía, devuelve la agenda igual a la resultante de eliminar de a el último
    contacto añadido a ella. Si a está vacía, devuelve la agenda vacía}
  está: agenda a, contacto c -> booleano
    {Dada una agenda a y un contacto c, devuelve verdad si y sólo si en
    a hay algún contacto igual a c (en el sentido de la función iguales del TAD contacto),
    falso en caso contrario}
  {EN LA IMPLEMENTACIÓN, AÑADIR LAS OPERACIONES DE UN ITERADOR1}
fespec

```

- c) Crea un pequeño programa principal en el que:

1. se introduzcan varios contactos en una agenda;
2. se muestren por pantalla los datos de todos los contactos introducidos¹;
3. se borre el último contacto, y se vuelvan a mostrar todos los datos¹;
4. dado el nombre de un contacto, compruebe si en la agenda hay algún contacto con ese nombre.

Instrucciones.

- El código fuente será compilado y probado en *lab000.unizar.es*, que es donde deberá funcionar correctamente.
- El código fuente deberá compilar correctamente con la opción `-std=c++11` activada.
 - Esto significa que, si se trabaja con la línea de comandos en *lab000*, deberá compilarse con (sin tener que indicar subdirectorios o rutas relativas para los ficheros):


```
g++ -std=c++11 ficheros_compilar...
```

- Si se trabaja con algún entorno de programación (*Geany*, *Visual Studio Code*, *Code::Blocks*, *CodeLite*, etc) y no se utiliza la línea de comandos para compilar, el proyecto de la práctica deberá estar configurado para compilar con la opción `-std=c++11`.
- Todos los ficheros con código fuente deberán estar **correctamente documentados**.
- En el comentario inicial de **cada fichero** de código fuente se añadirán los **nombres completos y NIP de los autores**.
- **Los TADs deberán implementarse siguiendo las instrucciones dadas en las clases y prácticas de la asignatura, y no se permite utilizar Programación Orientada a Objetos.**
- No se permite usar las clases o componentes de la *Standard Template Library (STL)*, ni similares.

¹ El texto subrayado (implementación de un iterador y muestra en pantalla de todos los datos) lo podréis realizar después de que veamos en clase el concepto de *iterador*.